

1 Solution — GIAM 2.7

Problem 2

1.1 Problem

Below is a rule of inference we call **extended elimination**:

$$\frac{\begin{array}{c} (A \vee B) \vee C \\ \neg A \\ \neg B \end{array}}{\therefore C}$$

Use a truth table to verify that this rule is valid.

1.2 The Truth Table

A rule is **valid** when no row makes every premise true while the conclusion is false — equivalently, when $\left[((A \vee B) \vee C) \wedge \neg A \wedge \neg B \right] \Rightarrow C$ is a tautology.

A	B	C	$A \vee B$	$(A \vee B) \vee C$	$\neg A$	$\neg B$	all premises?	C	rule
T	T	T	T	T	F	F	no	T	T
T	T	F	T	T	F	F	no	F	T
T	F	T	T	T	F	T	no	T	T
T	F	F	T	T	F	T	no	F	T
F	T	T	T	T	T	F	no	T	T
F	T	F	T	T	T	F	no	F	T
F	F	T	F	T	T	T	YES	T	T
F	F	F	F	F	T	T	no	F	T

Table 1.1

The final column is **T** in all eight rows, so the rule is **valid**. ■

Valid means: no row has every premise true and the conclusion false. Only one row here even has all three premises true.

A	B	C	$A \vee B$	$(A \vee B) \vee C$	$\neg A$	$\neg B$	all premises?	C	rule
T	T	T	T	T	F	F	no	T	T
T	T	F	T	T	F	F	no	F	T
T	F	T	T	T	F	T	no	T	T
T	F	F	T	T	F	T	no	F	T
F	T	T	T	T	T	F	no	T	T
F	T	F	T	T	T	F	no	F	T
F	F	T	F	T	T	T	YES	T	T
F	F	F	F	F	T	T	no	F	T

The rule column is T in all eight rows — the rule is VALID.

Seven rows pass for free because a premise fails; only the shaded row does real work, and there C is true.

Route 1 — regroup, then eliminate twice $(A \vee B) \vee C \equiv A \vee (B \vee C)$ <i>associativity</i> with $\neg A \Rightarrow B \vee C$ <i>disjunctive syllogism</i> with $\neg B \Rightarrow C$ <i>disjunctive syllogism</i> strip one disjunct at a time	Route 2 — collapse the pair first $\neg A \wedge \neg B \equiv \neg(A \vee B)$ <i>De Morgan</i> write $X = A \vee B$, so we have $X \vee C$ with $\neg X \Rightarrow C$ <i>disjunctive syllogism</i> one elimination on a compound disjunct
It generalises: from $A_1 \vee A_2 \vee \dots \vee A_n$ together with $\neg A_1, \dots, \neg A_{n-1}$, conclude A_n. <i>This repeated stripping is exactly "unit propagation", the innermost loop of every modern SAT solver.</i>	

Figure 1.1: An eight-row truth table with columns A, B, C, A or B, (A or B) or C, not-A, not-B, an ‘all premises?’ column, C, and the rule column. In seven rows the ‘all premises’ column reads no, because at least one premise fails. Exactly one row — A false, B false, C true — is shaded and reads YES, and in that row C is true. The rule column is T in all eight rows, so the rule is valid; seven rows pass for free because a premise fails, and only the shaded row does real work. Below, two derivations. Route 1, regroup then eliminate twice: (A or B) or C is equivalent to A or (B or C) by associativity; with not-A this gives B or C by disjunctive syllogism; with not-B this gives C by disjunctive syllogism again — stripping one disjunct at a time. Route 2, collapse the pair first: not-A and not-B is equivalent to not(A or B) by De Morgan; writing X for A or B, we have X or C together with not-X, which gives C by a single disjunctive syllogism on a compound disjunct. A closing note observes the rule generalises: from a disjunction of n things together with the negations of the first n minus one, conclude the last — which is exactly unit propagation, the innermost loop of every modern SAT solver.

1.3 Only One Row Does Any Work

Notice how lopsided the table is. In **seven** of the eight rows at least one premise is false, so the conditional is **vacuously true** (Problem 2.2.6) and tells us nothing. The rule's entire content sits in the single row

$$A = \text{F}, \quad B = \text{F}, \quad C = \text{T},$$

which is the only assignment satisfying all three premises — and there the conclusion C is indeed true. This is worth noticing as a technique: to check a rule you need only find the rows where **all premises hold** and confirm the conclusion in each. The other rows are guaranteed to pass.

Why is that the only such row? The premises $\neg A$ and $\neg B$ force $A = B = \text{F}$, which collapses $(A \vee B) \vee C$ to just C ; the first premise then forces $C = \text{T}$.

1.4 Two Derivations From Rules Already Known

The rule is not new information — it follows from tools already in hand, in two different ways.

Route 1 — regroup, then eliminate twice.

$$\begin{array}{ll} (A \vee B) \vee C \cong A \vee (B \vee C) & \text{associativity} \\ \text{with } \neg A \implies B \vee C & \text{disjunctive syllogism} \\ \text{with } \neg B \implies C & \text{disjunctive syllogism} \end{array}$$

Route 2 — collapse the pair first. By De Morgan, $\neg A \wedge \neg B \cong \neg(A \vee B)$. Writing $X = A \vee B$, the premises become $X \vee C$ and $\neg X$, and a single disjunctive syllogism yields C .

Both routes were checked; either is a complete justification, and the truth table confirms the result independently.

1.5 Remark — The Rule Generalises to Any Length

Nothing about the argument depends on there being three disjuncts. For any n :

$$\frac{A_1 \vee A_2 \vee \cdots \vee A_n \quad \neg A_1, \neg A_2, \dots, \neg A_{n-1}}{\therefore A_n}$$

Route 1 explains why: each $\neg A_i$ strips one disjunct by disjunctive syllogism, and after $n - 1$ strips only A_n remains. Extended elimination is just disjunctive syllogism iterated, which is why it deserves the name “extended” rather than a place in the basic catalogue. It is the formal version of the everyday process of elimination — rule out every suspect but one, and you have your answer.

1.6 Remark — Why Parenthesisation Does Not Matter Here

The rule is stated with a specific grouping, $(A \vee B) \vee C$, but the parentheses are cosmetic: $\vee$ is **associative**, so $(A \vee B) \vee C \cong A \vee (B \vee C)$ and both mean “at least one of the three.” Contrast this with Problem 2.2.3, where $\Rightarrow$ turned out *not* to be associative and the parentheses in $A \Rightarrow (B \Rightarrow C)$ genuinely mattered. Here we may drop them and write $A \vee B \vee C$ without ambiguity — which is exactly what makes the n -fold generalisation above well posed.

1.7 Remark — This Is Unit Propagation

The iterated form is not merely a curiosity; it is the workhorse of automated reasoning. SAT solvers store a formula as a set of **clauses** (disjunctions of literals), and the central operation of the DPLL algorithm is:

if every literal in a clause but one has been falsified, the remaining literal must be true.

That is precisely extended elimination, and it is called **unit propagation**. Modern solvers spend the large majority of their running time doing it, which is why the rule is implemented with great care. The humble process of elimination, formalised and run billions of times a second, is what lets a SAT solver handle formulas with millions of variables.

1.8 Remark — Checking Validity: Two Equivalent Framings

Two descriptions of validity were used above and it is worth seeing that they agree:

1. **No bad row.** There is no assignment making all premises true and the conclusion false.
2. **The conditional is a tautology.** $(\text{premises}) \Rightarrow (\text{conclusion})$ is true in every row.

These say the same thing, because a conditional is false in exactly one circumstance — true antecedent, false consequent (Problem 2.2.2). So “no bad row” and “no false entry in the final column” are the same check. Framing 1 is faster in practice: scan for rows where the premises all hold and inspect only those. With three premises that pruned eight rows down to one.